INTELLIGENT RAILWAY TRACK SURFACE DEFECT DETECTION

Mini Project Guide • ECE / Computer Vision / Machine Learning

Project type	Image-based defect detection prototype
Main tools	Python, OpenCV, TensorFlow/Keras or PyTorch
Input	Railway-track surface images
Output	Predicted class: Defect / No Defect (or defect categories)

Purpose: This guide explains how to plan, build, test, and present a student prototype. It is not a certified railway inspection system; real railway safety decisions require qualified inspection and validated equipment.

Prepared as a step-by-step learning guide

1. Abstract

Railway tracks are exposed to wear, environmental conditions, and repeated loads. Surface problems such as cracks, corrosion, unusual wear, and visible damage should be identified early so trained maintenance teams can investigate them. This project proposes an image-based prototype that uses computer vision and machine learning to classify railway-track surface images as “Defect” or “No Defect.” The system accepts an image, resizes and normalizes it, runs inference through a trained model, and displays the predicted class with a confidence score. It can be extended to multiple defect categories and a simple web interface. Its results are decision support only, not proof that a track is safe.

2. Problem Statement

Manual visual inspection can be time-consuming and may vary with lighting, viewing angle, image quality, and inspector experience. The goal is to develop a student-level tool that analyses supplied railway-track images and flags images that may contain visible surface defects for further review.

3. Objectives

• Collect and organize labelled railway-track surface images.

• Pre-process images into a consistent size and pixel range.

• Train or fine-tune an image-classification model.

• Predict whether an input image appears defective or non-defective.

• Evaluate the model using a held-out test set and report limitations.

• Build a simple interface for uploading an image and viewing the result.

4. Scope and Limitations

• Scope: visible surface-image classification using a labelled dataset.

• The first version uses two classes: Defect and No Defect.

• Possible later classes: crack, corrosion, surface wear, and other visible damage—only if reliable labels are available.

• A photo may miss internal rail flaws, hidden damage, or defects outside the camera view.

• Performance can change with lighting, blur, camera angle, rail type, and dataset bias.

• Do not use the student prototype to make operational railway-safety decisions.

5. Proposed System Architecture

Image dataset → **Train/validation/test split** → **Image preprocessing** → **Model training** → **Model evaluation** → **Save model** → **Upload test image** → **Prediction and result display**

Suggested modules

• **Dataset module:** stores images in class folders and checks labels.

• **Preprocessing module:** resizes images (for example, 224 × 224 pixels) and applies model-appropriate normalization.

• **Training module:** trains a CNN or fine-tunes a pre-trained model such as MobileNetV2.

• **Evaluation module:** reports accuracy, precision, recall, F1-score, and a confusion matrix.

• **Prediction module:** loads a saved model and predicts the class of a new image.

• **User interface:** optionally uses Streamlit to upload and display an image and prediction.

6. Software and Hardware Requirements

Category	Recommended
Language	Python 3.10 or a version supported by the selected ML library
Libraries	TensorFlow/Keras or PyTorch, OpenCV, NumPy, Matplotlib, scikit-learn
Optional UI	Streamlit
Development environment	Google Colab or a computer with Python installed
Hardware	4–8 GB RAM recommended; GPU helpful but not essential for a small transfer-learning experiment
Data	A properly licensed, labelled dataset of railway-track surface images

7. Dataset Preparation

Choose a dataset that explicitly permits your intended educational use. Read its licence, citation requirements, label definitions, and collection conditions. Do not download random images and assume their labels are reliable.

• Create class folders such as dataset/train/Defect and dataset/train/No_Defect, plus matching validation and test folders.

• Keep training, validation, and test images separate. If several images come from the same video or track segment, keep related images in the same split to reduce data leakage.

• Remove duplicates, unreadable files, and images with uncertain labels.

• Aim for balanced classes where possible and document the number of images in each split.

• Use augmentation only on training images, such as small rotations or brightness changes, if appropriate.

8. Implementation Plan

Step 1 — Set up the environment

Use Google Colab or local Python. Install required packages and confirm the ML framework imports successfully.

Step 2 — Arrange the data

Create train, validation, and test folders. Check that every image has a reliable label.

Step 3 — Preprocess images

Resize all images to the model input size and apply the preprocessing expected by the selected model.

Step 4 — Train a baseline

Start with transfer learning (for example, MobileNetV2) or a small CNN. Use the validation set to monitor overfitting.

Step 5 — Evaluate

Run the final model on the test set after model selection. Report accuracy, precision, recall, F1-score, and the confusion matrix.

Step 6 — Save the model

Save the trained model and the class-name order used during training.

Step 7 — Build prediction UI

Optionally create a Streamlit page to upload a photo and show the predicted class and confidence.

Step 8 — Document results

Record dataset source, splits, settings, metrics, sample predictions, errors, and limitations.

9. Starter Code (TensorFlow/Keras)

This is a template for a folder-based dataset. Update directory paths and check the model's preprocessing requirements before running it. A suitable dataset is required; this code is not a trained system by itself.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

IMG_SIZE = (224, 224)
BATCH_SIZE = 32
DATA_DIR = "dataset"

train_ds = keras.utils.image_dataset_from_directory(
f"{DATA_DIR}/train", image_size=IMG_SIZE,
batch_size=BATCH_SIZE, label_mode="binary"
)
val_ds = keras.utils.image_dataset_from_directory(
f"{DATA_DIR}/val", image_size=IMG_SIZE,
batch_size=BATCH_SIZE, label_mode="binary"
)

base = keras.applications.MobileNetV2(
input_shape=IMG_SIZE + (3,), include_top=False,
weights="imagenet"
)
base.trainable = False

inputs = keras.Input(shape=IMG_SIZE + (3,))
x = keras.applications.mobilenet_v2.preprocess_input(inputs)
x = base(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)

model.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["accuracy", keras.metrics.Precision(),
keras.metrics.Recall()])
model.fit(train_ds, validation_data=val_ds, epochs=5)
model.save("rail_track_defect_model.keras")
```

Important: Check the class order returned by the dataset loader and map model output to the correct class name. Add a separate test dataset and evaluate it before reporting results. The sample code is a starting point, not evidence of accuracy.

10. Example Prediction Logic

```
import numpy as np
from tensorflow import keras

model = keras.models.load_model("rail_track_defect_model.keras")
img = keras.utils.load_img("sample_track.jpg", target_size=(224, 224))
arr = keras.utils.img_to_array(img)
arr = np.expand_dims(arr, axis=0)

score = float(model.predict(arr, verbose=0)[0][0])
print("Model score:", score)
# Map score to a class only after confirming class-label order.
```

Use exactly the same preprocessing during training and prediction, and show a caution note that uncertain predictions need human review.

11. Evaluation

• **Accuracy:** fraction of all test predictions that are correct.

• **Precision:** among images predicted as defective, the fraction that are truly defective.

• **Recall:** among truly defective images, the fraction detected by the model.

• **F1-score:** harmonic mean of precision and recall.

• **Confusion matrix:** counts correct and incorrect predictions for each class.

Never invent performance numbers. Run evaluation on a separate test set and report measured results, class balance, and failure examples. In safety-related tasks, missed defects are particularly important to discuss.

12. Expected Output

When a user uploads a railway-track image, the interface should display the image, predicted class (Defect or No Defect), and a score with a caution note. Placeholder only: “Prediction: Defect — score: [measured value]. Needs inspection by a qualified person.” Replace placeholders with actual test results.

13. Future Enhancements

• Train a multi-class model for specific visible defect categories.

• Use object detection or segmentation to highlight the suspected defect region.

• Add image-quality checks and an “uncertain—manual review” outcome.

• Evaluate across different locations, weather, lighting, cameras, and rail conditions.

• Create a dashboard to record image IDs, predictions, and human-review outcomes.

14. Conclusion

This project provides a practical introduction to computer vision and machine learning through railway-track surface-image classification. The workflow covers dataset preparation, preprocessing, model training, evaluation, and a simple prediction interface. Dataset quality and independent testing are essential. The prototype can flag images for further review, but it cannot replace certified inspection, engineering judgement, or railway maintenance procedures.

15. Viva / Presentation Questions

1. What is the aim of the project?

To classify railway-track surface images as potentially defective or non-defective using computer vision and machine learning.

2. Why is image preprocessing required?

It standardizes image size and pixel representation so the model can process images consistently.

3. What is a CNN?

A Convolutional Neural Network is a deep-learning model commonly used to learn patterns from images.

4. What is transfer learning?

It reuses a model trained on a large dataset and adapts it to a new task.

5. Why split the dataset?

Training fits the model, validation helps select settings, and testing estimates performance on unseen data.

6. Why is accuracy alone insufficient?

It can hide poor performance on a minority class; precision, recall, F1-score, and a confusion matrix add detail.

7. What are the limitations?

Image quality, lighting, dataset bias, label errors, and hidden/internal flaws can affect reliability.

8. Can this system certify a railway track as safe?

No. It is an educational prototype and requires expert inspection and rigorous validation before operational use.

16. Suggested Report Format

• Cover page

• Certificate / declaration (only if required by your college)

• Acknowledgement

• Abstract

• Introduction and problem statement

• Literature review

• Existing system and proposed system

• System architecture and methodology

• Dataset and implementation

• Results and discussion

• Limitations and future scope

• Conclusion and references

Before submission, add your dataset source and licence, screenshots of your running implementation, actual measured metrics, and your department's required format. Do not claim experiments were completed unless you ran them.

17. Short Presentation Summary

“Our project is Intelligent Railway Track Surface Defect Detection. The goal is to analyse railway-track surface images and identify images that may contain visible defects. We use Python, image preprocessing, and a machine-learning image classifier. The dataset is divided into training, validation, and test sets. After training, the model predicts Defect or No Defect for a new image. We evaluate the system using accuracy, precision, recall, F1-score, and a confusion matrix. This is a student prototype that can support further review, but it cannot replace qualified railway inspection.”